

Vision-Based Maze Solving with a 4-DOF Manipulator

Valentin Gaucher*

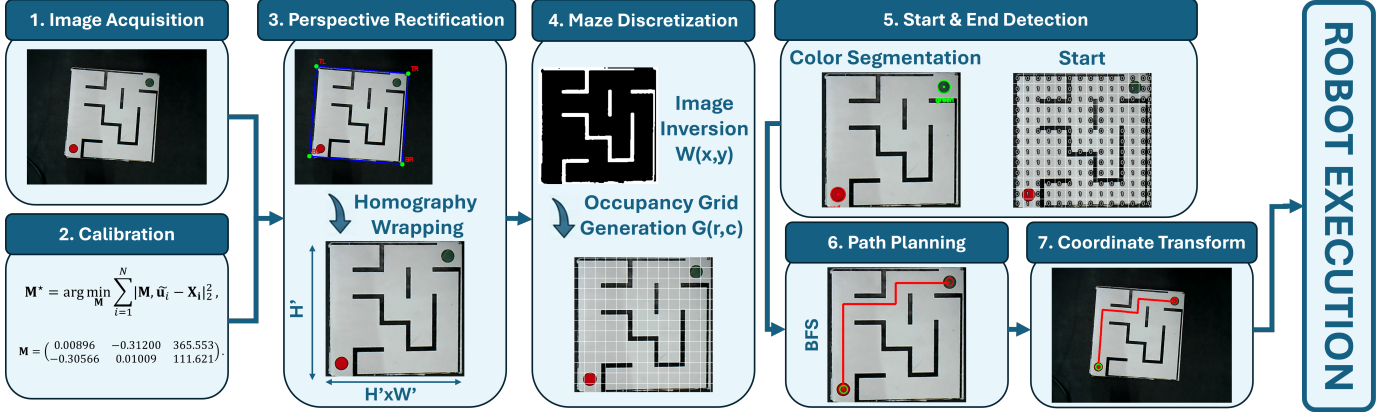


Fig. 1: End-to-end processing pipeline for the robotic maze-solving system. The process flows from initial image acquisition and extrinsic calibration (Parts 1–2) through vision-based perception (image rectification, maze discretization, and marker detection). Once the workspace is mapped to a digital grid, the optimal path is computed using Breadth-First Search (BFS) and translated via inverse kinematics and coordinate transformations into physical trajectories for the robotic manipulator.

Abstract—This report describes the design, implementation, and experimental validation of a vision-based maze-solving system employing a Dobot Magician 4-DOF robotic manipulator and a fixed overhead USB camera. The system addresses two coupled problems. First, the extrinsic calibration between camera pixel coordinates and the robot end-effector workspace is estimated through a planar affine transformation recovered from a set of manually annotated correspondences. Second, the maze is processed through a perception pipeline that detects the maze boundary, applies a perspective rectification, discretizes the workspace into an occupancy grid, identifies the start and end markers by color, and computes a collision-free path using breadth-first search (BFS). The resulting path is projected back into camera coordinates and subsequently transformed into robot coordinates for execution. Experimental results demonstrate that the full pipeline successfully solves a physical maze, with the reconstructed path matching the intended traversal from the red start marker to the green end marker. The video of the execution is available here https://youtube.com/shorts/IbocXOF_1cs?si=cs_2f7zfbckXeNQZ

I. INTRODUCTION

Robotic manipulation guided by visual perception is a canonical problem in robotics that exercises several foundational subsystems, including camera-to-robot extrinsic calibration, image-space geometric reasoning, discrete path planning, and closed-loop motion execution. The maze-solving task considered in this project, although conceptually simple, requires all of these components to operate consistently with one another. Any inconsistency in the coordinate frames, the wall discretization, or the path representation will manifest as

the end-effector missing the intended trajectory or colliding with obstacles.

The objective of this project is to design a system in which a Dobot Magician manipulator traces the solution path of an arbitrary planar maze placed within the camera field of view. The maze is printed on white paper with black walls and is marked with a red circle denoting the start position and a green circle denoting the goal position. The system is decomposed into two parts. Part 1 addresses the calibration between the overhead camera and the robot base frame [1]. Part 2 addresses the vision pipeline that recovers a pixel-space solution path from a captured image of the maze.

The remainder of this report is organized as follows. Section II presents the theoretical background required for the coordinate transformations, perspective rectification, discretization, and shortest-path computation. Section III describes the overall method and the responsibilities of each pipeline stage. Section IV reports the experimental procedure for both parts. Section V presents quantitative and qualitative results. Section VI discusses limitations and possible improvements. Section VII concludes.

A. A little history

The quest for robotic maze solving has a rich history, most notably popularized by the international Micromouse competition which began in the late 1970s. In this challenge, small autonomous robots are tasked with mapping and navigating a maze as rapidly as possible. A highly interesting and easy to watch video by Veritasium detail this history and the methods <https://www.youtube.com/watch?v=ZMQbHMgK2rw> [2]. While advanced micromice often employ dynamic algorithms like Flood Fill to explore and calculate the absolute fastest

route to the center on the fly, the foundational logic driving these solvers is deeply rooted in unweighted graph traversal techniques. This project draws inspiration from that legacy, applying Breadth-First Search (BFS) [3] to a vision-based overhead system to compute the optimal, collision-free path for a robotic manipulator to execute.

II. BACKGROUND

A. Camera Model and the Planar Assumption

A general projective camera is described by the pinhole model [4], which maps a point $\mathbf{X} = (X, Y, Z)^\top$ expressed in a world frame to a homogeneous image point $\tilde{\mathbf{x}} = (u, v, 1)^\top$ through

$$\lambda \tilde{\mathbf{x}} = \mathbf{K} [\mathbf{R} \mid \mathbf{t}] \tilde{\mathbf{X}}, \quad (1)$$

where $\mathbf{K}$ is the 3×3 intrinsic matrix, $[\mathbf{R} \mid \mathbf{t}]$ is the extrinsic transformation, $\tilde{\mathbf{X}}$ denotes homogeneous world coordinates, and λ is a nonzero scale factor.

In the present setup the maze lies on a planar workspace and the camera is fixed above this workspace. Under the planar assumption $Z = 0$, the mapping from the workspace plane to the image plane degenerates into a planar homography, which in its most general form is a 3×3 matrix $\mathbf{H}$ acting on homogeneous coordinates. When the camera optical axis is approximately perpendicular to the workspace and perspective effects are small, this homography can be approximated by a 2×3 affine transformation of the form

$$\begin{pmatrix} X \\ Y \end{pmatrix} = \mathbf{M} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \end{pmatrix} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}, \quad (2)$$

where (u, v) are pixel coordinates and (X, Y) are robot-frame coordinates on the workspace plane in millimeters.

The matrix $\mathbf{M}$ has six degrees of freedom and encodes rotation, anisotropic scaling, shear, and translation. A constrained variant known as the partial affine transformation eliminates the shear and anisotropic scaling terms, retaining only rotation, uniform scale, and translation. This partial transformation has four degrees of freedom and is parameterized by $s \cos \theta$, $s \sin \theta$, t_x , t_y , with

$$\mathbf{M}_{\text{partial}} = \begin{pmatrix} s \cos \theta & -s \sin \theta & t_x \\ s \sin \theta & s \cos \theta & t_y \end{pmatrix}. \quad (3)$$

Although the partial affine model is simpler and more robust to noise when only a few correspondences are available, it fails to capture the nonuniform scaling that arises when the camera is not perfectly orthogonal to the workspace. In Section V this limitation is demonstrated empirically.

B. Estimation of the Affine Transformation

Given N correspondences $\{(\mathbf{u}_i, \mathbf{X}_i)\}_{i=1}^N$, the affine matrix $\mathbf{M}$ is recovered by solving the linear least squares problem

$$\mathbf{M}^* = \arg \min_{\mathbf{M}} \sum_{i=1}^N \|\mathbf{M} \tilde{\mathbf{u}}_i - \mathbf{X}_i\|_2^2, \quad (4)$$

where $\tilde{\mathbf{u}}_i = (u_i, v_i, 1)^\top$. In practice this minimization is performed with outlier rejection via RANSAC, as implemented by the OpenCV functions `estimateAffine2D` and `estimateAffinePartial2D` [5]. The quality of the recovered matrix is assessed by the reprojection error

$$e_i = \|\mathbf{M} \tilde{\mathbf{u}}_i - \mathbf{X}_i\|_2, \quad (5)$$

which measures in millimeters the discrepancy between the transformed pixel correspondence and its known robot counterpart.

C. Perspective Rectification of the Maze

Although the affine model suffices for the pixel-to-robot mapping over a small workspace, the image of the maze itself is subject to a full projective distortion because the maze is generally neither centered nor axis-aligned in the image. To simplify the subsequent discretization, the region of interest bounded by the four corners $\{\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3, \mathbf{c}_4\}$ of the maze is rectified into a canonical axis-aligned rectangle through the planar homography $\mathbf{H}_{\text{rect}}$ that satisfies

$$\mathbf{H}_{\text{rect}} \tilde{\mathbf{c}}_i = \tilde{\mathbf{r}}_i, \quad i = 1, \dots, 4, \quad (6)$$

where $\tilde{\mathbf{r}}_i$ are the corners of the target rectangle of width W and height H , expressed in homogeneous coordinates. The homography is computed from four point correspondences using the direct linear transform [4]. Applying $\mathbf{H}_{\text{rect}}$ to the source image yields the warped image on which grid discretization is performed.

The inverse homography $\mathbf{H}_{\text{rect}}^{-1}$ subsequently maps points from the warped image back to the original camera frame, enabling the solution path computed in the warped space to be expressed in the original pixel coordinates from which the robot commands are ultimately derived.

D. Discretization and Occupancy Grid

After rectification, the warped maze image of dimensions $W \times H$ is partitioned into a regular grid of square cells of side length g pixels. The cell at grid coordinates (r, c) covers the pixel region

$$\Omega_{r,c} = \{(x, y) : cg \leq x < (c+1)g, rg \leq y < (r+1)g\}. \quad (7)$$

A binary wall mask $W(x, y)$ is obtained by applying an inverted Otsu threshold [6] to the grayscale image of the rectified maze. For each cell, the occupancy value $v_{r,c} \in \{0, 1\}$ is determined by the fraction of wall pixels within that cell,

$$v_{r,c} = \begin{cases} 0 & \text{if } \frac{1}{|\Omega_{r,c}|} \sum_{(x,y) \in \Omega_{r,c}} \mathbf{1}[W(x, y) > 0] \geq \tau, \\ 1 & \text{otherwise,} \end{cases} \quad (8)$$

where τ is a threshold expressed as a percentage and $\mathbf{1}[\cdot]$ is the indicator function. Cells with $v_{r,c} = 1$ are considered traversable while cells with $v_{r,c} = 0$ are considered walls.

E. Start and End Detection via Color Segmentation

The start and end markers are detected in two stages. Candidate circular regions are first localized by the Circular Hough Transform [7], which detects the parameters (x_c, y_c, r) of circles in an edge image. For each detected candidate, the dominant color within the circular region is classified by converting the image to the HSV color space and computing the ratio of pixels falling within predefined hue ranges for red and green. Because red hues wrap around in the HSV space, two disjoint intervals are employed to capture both the low-hue and high-hue red components. The classification rule for a color C is

$$C(c) = \begin{cases} \text{red} & \text{if } N_R(c) > N_G(c) \text{ and } N_R(c) > \alpha|c|, \\ \text{green} & \text{if } N_G(c) > N_R(c) \text{ and } N_G(c) > \alpha|c|, \\ \text{unknown} & \text{otherwise,} \end{cases} \quad (9)$$

where $N_R(c)$ and $N_G(c)$ denote the pixel counts within the respective color masks, $|c|$ is the area of the circular region, and α is a minimum density parameter.

F. Shortest Path via Breadth-First Search

Given the occupancy grid, the maze is modeled as an unweighted graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ in which each vertex $\mathcal{V}$ corresponds to a traversable cell and edges $\mathcal{E}$ connect 4-adjacent cells. Because all edges have unit weight, the shortest path between the start cell s and the end cell t is efficiently computed by breadth-first search [3], which has time complexity $\mathcal{O}(|\mathcal{V}| + |\mathcal{E}|)$ and guarantees optimality in the number of moves.

The admissibility of a vertex is governed by the rule that interior cells must satisfy $v_{r,c} = 1$, whereas the start and end cells may have $v_{r,c} = 0$ because the start and end markers occlude the underlying wall structure. The BFS procedure maintains a parent pointer for each visited vertex, allowing the path to be reconstructed by backward traversal from t to s once the goal is reached.

III. METHOD

The system is organized into two parts, each consisting of several sequential stages. An overview of the complete pipeline is given below.

A. Camera Calibration

Part 1 estimates the affine transformation $\mathbf{M}$ between pixel and robot coordinates. The procedure is as follows.

- 1) A black background is placed within the camera field of view, and the robot is homed.
- 2) A set of N well-distributed reference points is selected visually in the camera frame. For each point, the user records the pixel location (u_i, v_i) by clicking on the image and the corresponding robot position (X_i, Y_i) by commanding the end-effector to the same physical location and reading the reported pose.
- 3) The correspondences are written into a Python script, which computes $\mathbf{M}$ using the OpenCV function

`estimateAffine2D`. The full affine model (six degrees of freedom) is preferred over the partial affine model because the latter was shown empirically to produce unacceptably large reprojection errors for this setup.

- 4) The computed matrix is validated by evaluating Equation (5) on every calibration point. The matrix is accepted if the mean reprojection error is below a threshold of approximately 5 mm.
- 5) The validated matrix is copied into the motion execution script `maze_motion.py` as the operational transformation.

B. Vision Pipeline and Path Execution

Part 2 processes the captured maze image through a sequence of six stages, summarized in Table I.

Each stage reads its input from disk, performs its computation, and writes its output. This modular structure allows any stage to be tested in isolation and facilitates debugging.

IV. EXPERIMENTS

A. Hardware Setup

The experimental platform consists of a Dobot Magician manipulator connected over USB to a host computer on the serial port `/dev/ttyACM0`. A USB overhead camera captures frames at a resolution of 640×480 pixels using MJPG encoding. A planar maze printed in black ink on white paper is positioned on a dark background within the field of view of both the camera and the reachable workspace of the robot.

B. Part 1: Calibration Procedure

Eight correspondences were collected by selecting visible features of the maze and calibration pattern within the camera frame, then manually commanding the robot to each location and recording the reported Cartesian pose. The correspondences are listed in Table II.

The affine transformation was first estimated using the partial affine model `estimateAffinePartial2D`, and the reprojection errors were computed. The model was then re-estimated using the full affine model `estimateAffine2D` for comparison.

C. Part 2: Maze Solving Procedure

A physical maze printed and placed on a dark background was used as the test case. The captured image, intermediate representations, and final solution were written to the directory `part_2_maze_solution`.

V. RESULTS

A. Part 1: Calibration Accuracy

The partial affine model produced a matrix with mean reprojection error of approximately 65.7 mm and maximum error of approximately 201.5 mm. Upon inspection of the RANSAC inlier mask, only four of the eight correspondences were retained as inliers, and the remaining four were rejected

Stage	Script	Input	Output	Purpose
1	01_capture_image.py	Camera frame	Corners JSON	Detect maze boundary
2	02_maze_warp_from_json.py	Corners JSON, image	Warped image	Perspective rectification
3	03_maze_circles_and_grid.py	Warped image	Grid JSON, walls mask	Discretization and circle detection
4	04_solve_maze.py	Grid JSON	Path JSON	BFS shortest path
5	05_unwrap_and_overlay_path.py	Corners JSON, Path JSON	Unwarped path JSON	Map path to camera space
6	maze_motion.py	Unwarped path JSON	Robot motion	Execute on hardware

TABLE I: Pipeline stages and the data flow between them.

Point	u (px)	v (px)	X (mm)	Y (mm)
1	145	98	335.8	73.1
2	538	93	342.2	-51.6
3	489	121	331.3	-37.2
4	182	153	320.0	57.2
5	383	153	320.7	-3.9
6	238	221	297.8	41.7
7	337	221	300.3	11.5
8	280	288	278.5	28.3

TABLE II: Calibration correspondences between pixel coordinates and robot coordinates.

as outliers. This outcome indicates that the similarity transformation is insufficient to model the true geometric relationship between the camera and the robot, presumably because the optical axis of the camera is not exactly perpendicular to the workspace and thus introduces nonuniform scaling.

The full affine model, by contrast, produced a mean reprojection error of 1.34 mm and a maximum of 4.83 mm. All correspondences were retained as inliers under this model. The recovered matrix is

$$\mathbf{M} = \begin{pmatrix} 0.00896 & -0.31200 & 365.553 \\ -0.30566 & 0.01009 & 111.621 \end{pmatrix}. \quad (10)$$

These results are summarized in Table III. The full affine model was therefore adopted for all subsequent experiments.

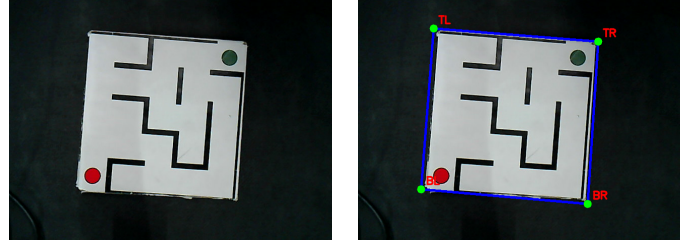
Model	Mean e (mm)	Max e (mm)
estimateAffinePartial2D	65.71	201.47
estimateAffine2D	1.34	4.83

TABLE III: Comparison of reprojection errors (e) for the two affine estimation methods on the eight calibration correspondences.

B. Part 2: Pipeline Outputs

The outputs of the six pipeline stages for a representative test maze are shown in Figures 2 through 5. The captured camera frame (Figure 2a) shows the maze placed on a dark background with the red start marker in the lower-left region and the green end marker in the upper-right region. The maze boundary was successfully detected as the largest quadrilateral in the thresholded image, as visualized by the overlay in Figure 2b.

The perspective rectification produced the axis-aligned image shown in Figure 3a. In this canonical frame the maze corners coincide with the corners of the image, which simplifies the subsequent grid discretization.



(a) Raw camera frame.

(b) Maze boundary and corners.

Fig. 2: Stage 1 (left) and the overlay produced by the corner detector (right). The four corners are ordered consistently as top-left, top-right, bottom-right, bottom-left.

The Circular Hough Transform correctly detected both the red and green markers, and the HSV color classification assigned the correct labels to each. The binary wall mask (Figure 3c) was obtained by inverted Otsu thresholding followed by explicit removal of the detected circles, ensuring that the start and end cells are not erroneously classified as walls.

The discretized occupancy grid is overlaid on the warped image in Figure 4a. Each cell is labeled with its value, where 1 denotes a traversable cell and 0 denotes a wall. The solution path computed by BFS is drawn on the warped image in Figure 4b, showing a traversal from the start cell near the red marker to the end cell near the green marker through a sequence of 4-connected moves.

Finally, the solution path is transformed back into the coordinate frame of the original camera image through the inverse homography, producing the overlay shown in Figure 5. The reconstructed path correctly follows the maze corridors in the original camera view, confirming that the forward and inverse rectifications are consistent.

The solution path contains 20 moves between 21 cells. When augmented with the start and end circle centers, the full pixel polyline has 23 waypoints, which are transformed by the affine matrix of Equation (10) and executed by the manipulator at a fixed tracing height.

VI. DISCUSSION

A. On the Choice of Affine Model

The most significant empirical finding of Part 1 is the pronounced difference in accuracy between the partial affine model and the full affine model. The partial affine transformation assumes that the camera sees the workspace under a pure similarity, that is, a rotation followed by a uniform scaling. In practice this assumption is violated when the camera is tilted with respect to the workspace plane, which

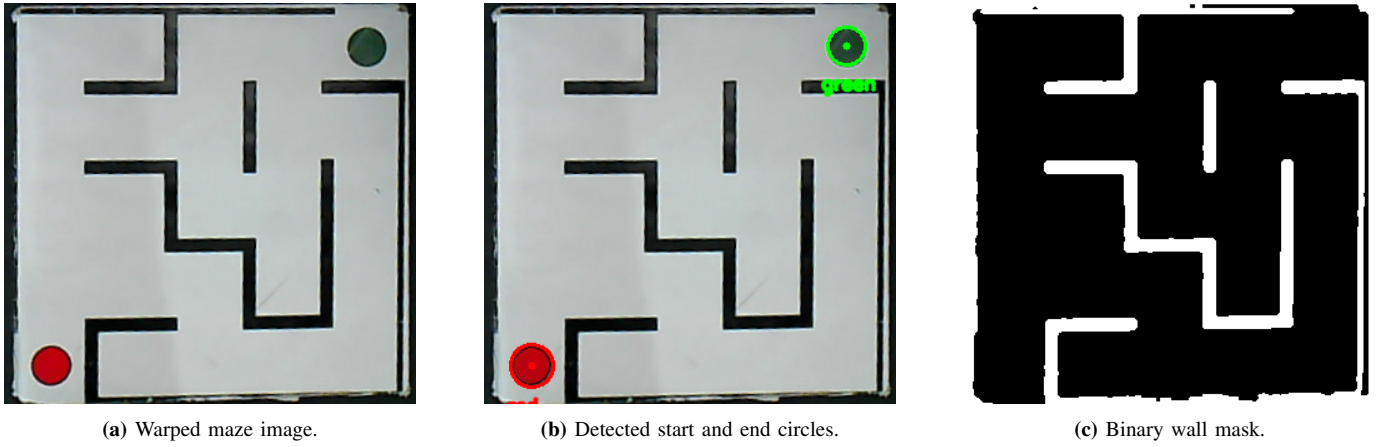


Fig. 3: Stages 2 and 3 of the pipeline. The warped image is processed to identify the start and end markers by color, and the wall structure is isolated by thresholding.

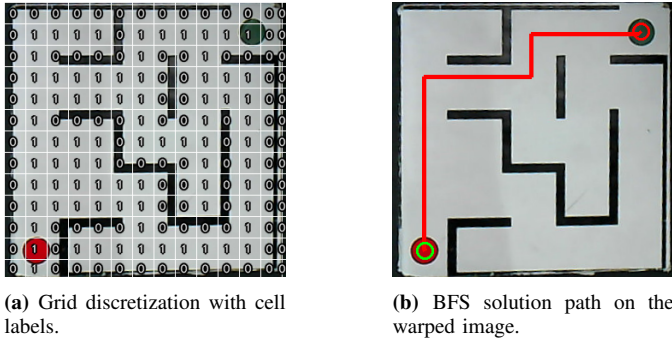


Fig. 4: Stage 3 (grid discretization) and Stage 4 (path planning). The BFS solution contains 20 moves across 21 traversable cells.

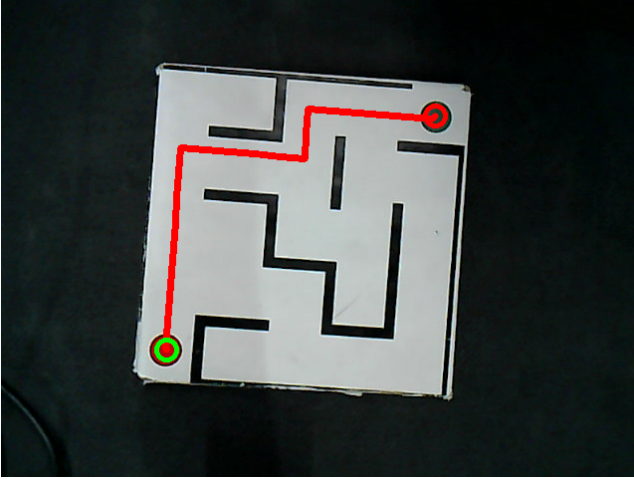


Fig. 5: Stage 5 output. The BFS path computed in the warped space has been mapped back to the original camera image using the inverse of the rectification homography. This is the set of pixel waypoints that is subsequently transformed into robot coordinates and executed by the manipulator.

introduces a foreshortening effect along one axis. The residual error produced by forcing a uniform scale onto a nonuniform mapping is large, and because the RANSAC procedure rejects the most discrepant points as outliers, the resulting model is fit to only a subset of the data and is effectively unusable outside

that subset.

The full affine model accommodates the missing degrees of freedom at no additional computational cost. It remains, however, an approximation to the full perspective projection. For applications in which the workspace is substantially larger or the camera is tilted more severely, a homography estimated by `findHomography` would be appropriate.

B. On the Grid Discretization

The choice of the grid cell size g governs a tradeoff between spatial resolution and robustness. A small g produces a fine grid that captures narrow passages but is sensitive to noise and imperfect wall alignment. A large g aggregates pixel noise and yields a more robust classification at the cost of occasionally merging adjacent passages. In the present experiments a value of $g = 25$ pixels was used with a threshold $\tau = 10\%$, which produced correct wall classification for the test maze. For denser mazes or finer walls, the grid parameters would require retuning.

C. On the End-to-End Consistency

The successful execution of the pipeline depends critically on the consistency of the coordinate frames at every transition. The corners detected in Stage 1 define the rectification homography used in Stage 2, and the same corners must be used in Stage 5 to invert that homography. Similarly, the grid centers computed in Stage 3 must be interpreted in the warped frame, and they must be transformed through the same homography before being expressed in the original image. Any drift in the reference corners between these stages would compound and misalign the final path.

D. Limitations

Several limitations of the current implementation are worth noting. The manual collection of calibration correspondences is laborious and error-prone. An automated calibration procedure employing a printed pattern would provide more precise and more numerous correspondences. The Circular Hough Transform requires careful parameter tuning and may fail

under adverse lighting conditions. The BFS shortest path minimizes the number of discrete moves but produces axis-aligned trajectories with sharp corners, which may be suboptimal for smooth robot motion. A post-processing step such as path smoothing or replacement by a B-spline would yield trajectories that are kinematically gentler.

VII. CONCLUSION

This project integrated camera calibration, image rectification, grid discretization, shortest-path planning, and robot motion execution into a coherent vision-based maze-solving system. The Part 1 calibration demonstrated that the full affine model yields acceptable accuracy on the given correspondences (mean error 1.34 mm), whereas the partial affine model produces errors that are one to two orders of magnitude larger and is therefore unsuitable for the present geometry. The Part 2 vision pipeline successfully detects the maze boundary, rectifies the image, identifies the start and end markers by color, discretizes the workspace into an occupancy grid, computes a shortest path by breadth-first search, and projects the path back into the original camera coordinate frame. The resulting path correctly connects the red start marker to the green end marker through the maze corridors in the test image. The end-to-end consistency of the coordinate transformations and the modular design of the pipeline facilitated both debugging and successful physical execution on the Dobot Magician manipulator.

REFERENCES

- 1 Craig, J. J., *Introduction to Robotics: Mechanics and Control*, 4th ed. Pearson, 2018.
- 2 Muller, D., "The fastest maze-solving competition on earth," <https://www.youtube.com/watch?v=ZMQbHMgK2rw>, 2023, youTube channel: Veritasium.
- 3 Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
- 4 Szeliski, R., *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- 5 Bradski, G., "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.
- 6 Otsu, N., "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- 7 Duda, R. O. and Hart, P. E., "Use of the hough transformation to detect lines and curves in pictures," *Communications of the ACM*, vol. 15, no. 1, pp. 11–15, 1972.